CSE 2246
PS#3

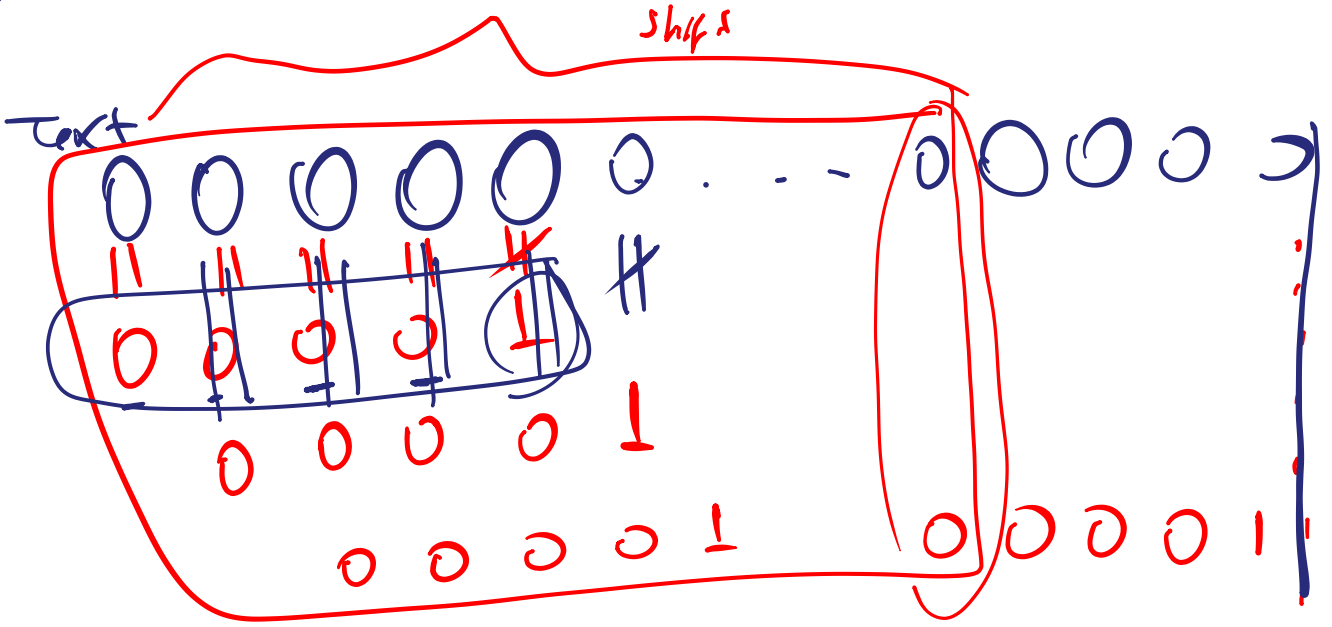
3.2

5. How many comparisons (both successful and unsuccessful) will be made by the brute-force algorithm in searching for each of the following patterns in the binary text of one thousand zeros?

a. 00001 **b.** 10000 **c.** 01010

Text : 0 0 0 0 . . . . 0 0 0

a)



shift = text leyt − pattern length + 1

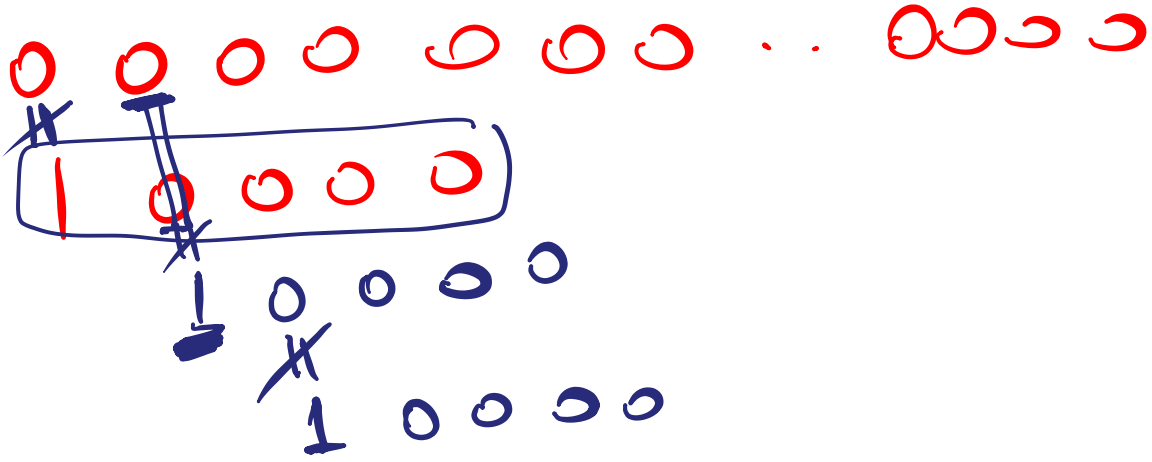
1000 − 5 + 1 = 996 //

In each run = 4 successful + 1 successful //

= 5

Total # of comparisons = 5 × 996 = 4980

b)

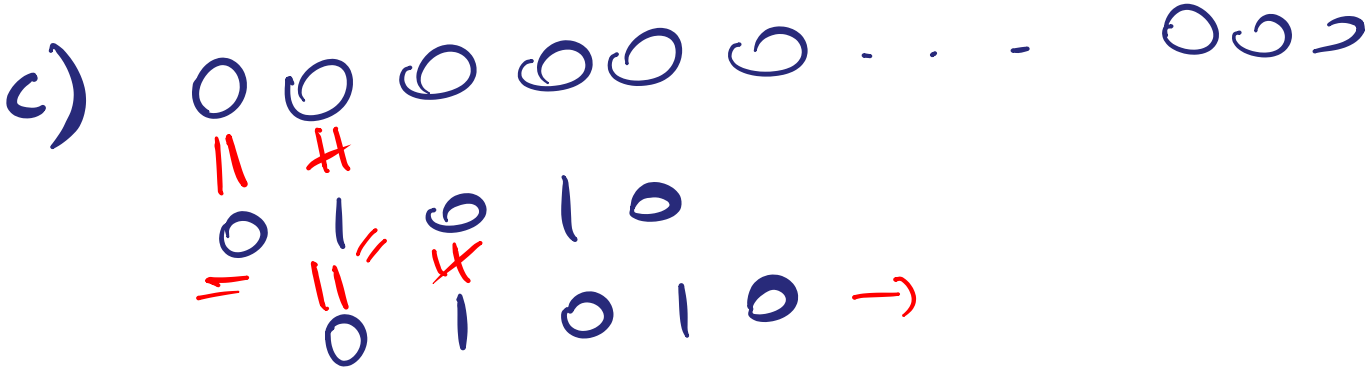


trial = text lenght − pattern +1 = 996

comparison in trial = 1

Comparison = 996 × 1 = 996

c)



trials = 996

comparison in each trial = 2

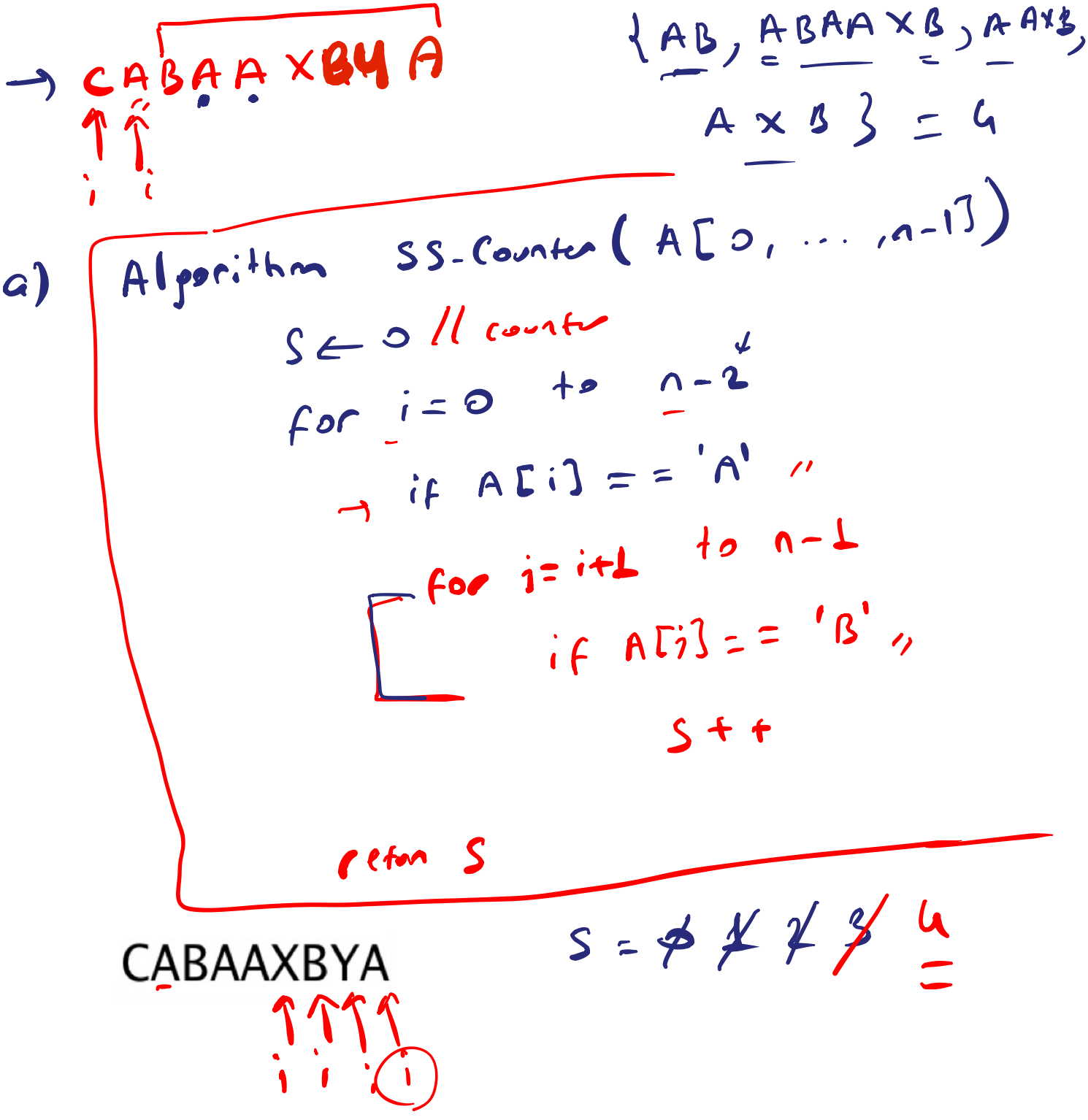
Comparison = 2 × 996 = 1992

3.2

8. Consider the problem of counting, in a given text, the number of substrings that start with an A and end with a B. For example, there are four such substrings in CABAAXBYA.

a. Design a brute-force algorithm for this problem and determine its efficiency class.

b. Design a more efficient algorithm for this problem. [Gin04]



txt: B O B - - - B B

$$C_{best} = \sum_{i=0}^{n-2} 1 = [n-2-0+1] = n-1$$

$$\Theta(n) //$$

---

txt: A A A A A A - - - A

$$C_{worst} = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 2 \approx \Theta(n^2) //$$

General time complexity : $C = O(n^2)$

---

b)

Algorithm $SS_2(A[0, \ldots, n-1])$

Counter_A ← 0.

Counter_S ← 0.

→ for $i = 0$ to $n-1$

- if $A[i] == 'A'$

  Counter_A ++

- if $A[i] == 'B'$

  Counter_S ← Counter_S + Counter_A

return Counter_S

$\Theta(n)$

$\sum_{i=0}^{n-2} (1) = \Theta(n)$

CABAAXBYA

i

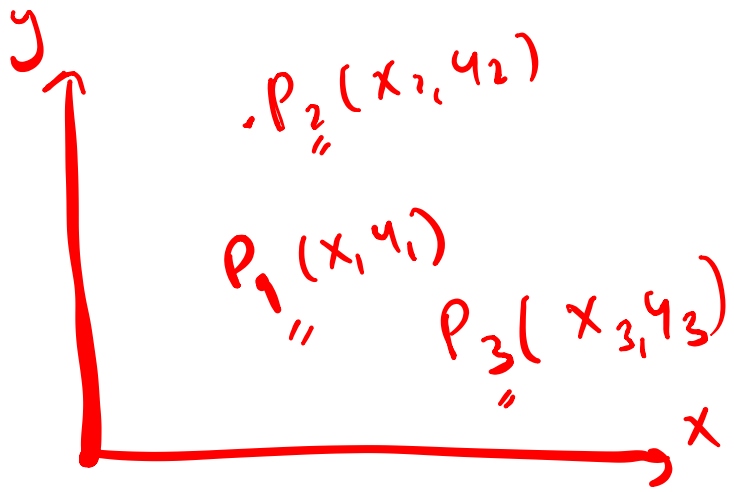
counter_A	Count_S
∅	0 "
X "	0+1 = 1
X	43 = 4
X Y "	=
4 "	

3.3

7. The closest-pair problem can be posed in the k-dimensional space, in which the Euclidean distance between two points $p'(x'_1, \ldots, x'_k)$ and $p''(x''_1, \ldots, x''_k)$ is defined as

$$d(p', p'') = \sqrt{\sum_{s=1}^{k=2} (x'_s - x''_s)^2}.$$

What is the time-efficiency class of the brute-force algorithm for the k-dimensional closest-pair problem?



$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$d_{13} = \sqrt{(x_1 - x_3)^2 + (y_1 - y_3)^2}$$

$$d_{23} = \sqrt{(x_2 - x_3)^2 + (y_2 - y_3)^2}$$

$$d = \min\{d_{12}, d_{23}, d_{13}\}$$

Algorithm CPP-2D($P_1, \ldots, P_n$)

$d \leftarrow \infty$

for $i = 0$ to $n-1$

  for $j = i+1$ to n

    $d_temp \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$

    $d \leftarrow \min\{d, d_temp\}$

return d

$$d(P', P'')$$

$$\| \quad \|$$

$$d(P^1, P^2) = \sqrt{\sum_{s=1}^{k} (x_s^1 - x_s^2)^2}$$

$$= \sqrt{(x_1^1 - x_1^2)^2 + (x_2^1 - x_2^2)^2}$$

$$\| \quad \|$$

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Algorithm CPP-KD($P_1, \ldots, P_n$)

$d \leftarrow \infty$

for $i = 0$ to $n-1$

for $j = i+1$ to n

$d_temp \leftarrow 0$

for $s \leftarrow 1$ to k

$d_temp \leftarrow d_temp + (x_s^i - x_s^j)^2$

$d \leftarrow \min\{d, \text{sqrt}(d_temp)\}$

return d

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n} \sum_{s=1}^{k} 1 = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (k)$$

$k \cdot [n - i - 1 + 1]$

$$= \sum_{i=0}^{n-1} k \cdot n - ki$$

$$= k \sum_{i=0}^{n-1} (n - i)$$

$$= k \left[(n-1-0) + (n-1) + (n-2) + \cdots (n-n+1) \right]$$

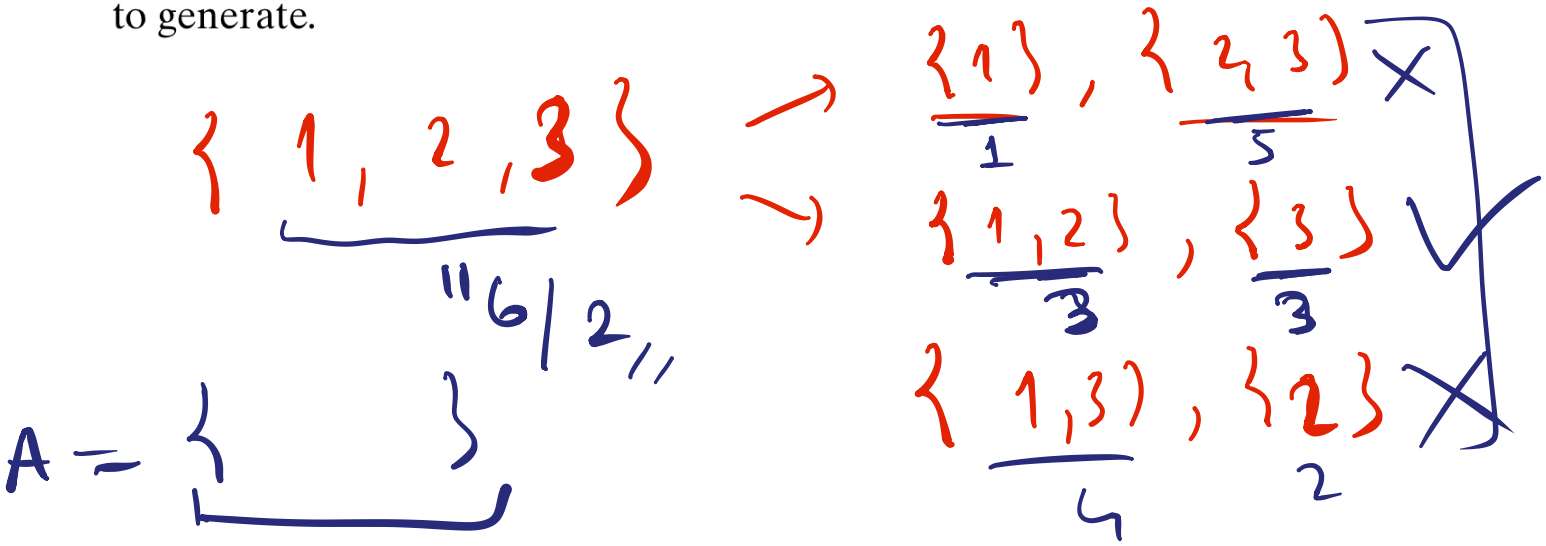
$$= k \cdot \left[(n-1) + n-2 + \cdots + 1 \right]$$

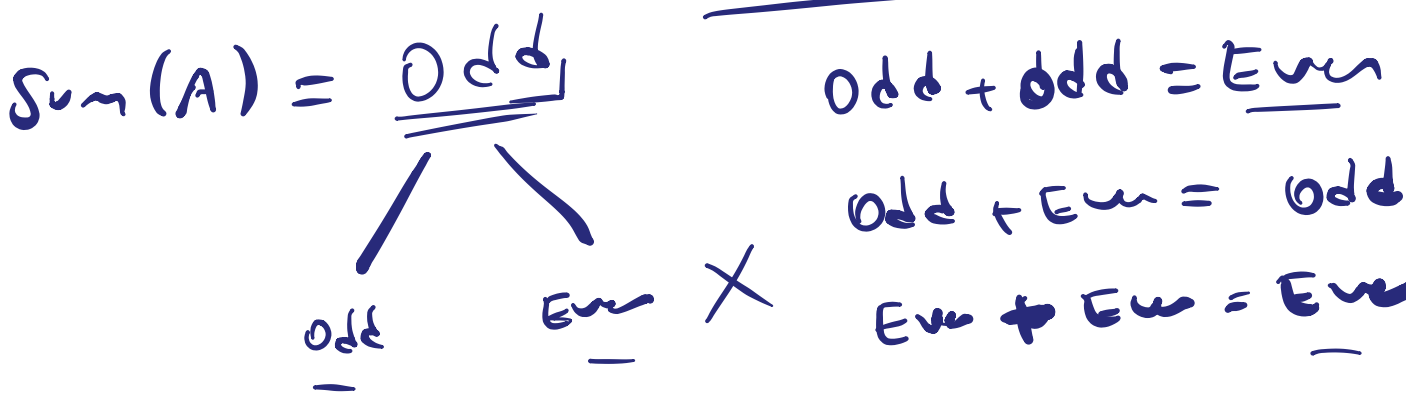
$$= k \cdot \frac{(n-1)(n-1+1)}{2} \simeq \frac{k \cdot n^2}{2}$$

$$\Theta (k \cdot n^2)$$

3.4

6. Consider the ***partition problem***: given n positive integers, partition them into two disjoint subsets with the same sum of their elements. (Of course, the problem does not always have a solution.) Design an exhaustive-search algorithm for this problem. Try to minimize the number of subsets the algorithm needs to generate.







- Compute sum S of given numbers

- If S is ODD, STOP (no solution)

- If S is EVEN,
  - Generate subsets until either a subset whose element's sum is S/2 is encountered or no more subsets are left

$\{5, 2, 1\} \rightarrow 8 \rightarrow \frac{8}{2} = 4$

$\{5\}$ $\{2, 1\}$ ×

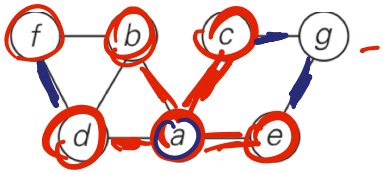
$\{5, 1\}$, $\{2\}$ ×

$\{5, 2\}$, $\{1\}$ ×

No solution

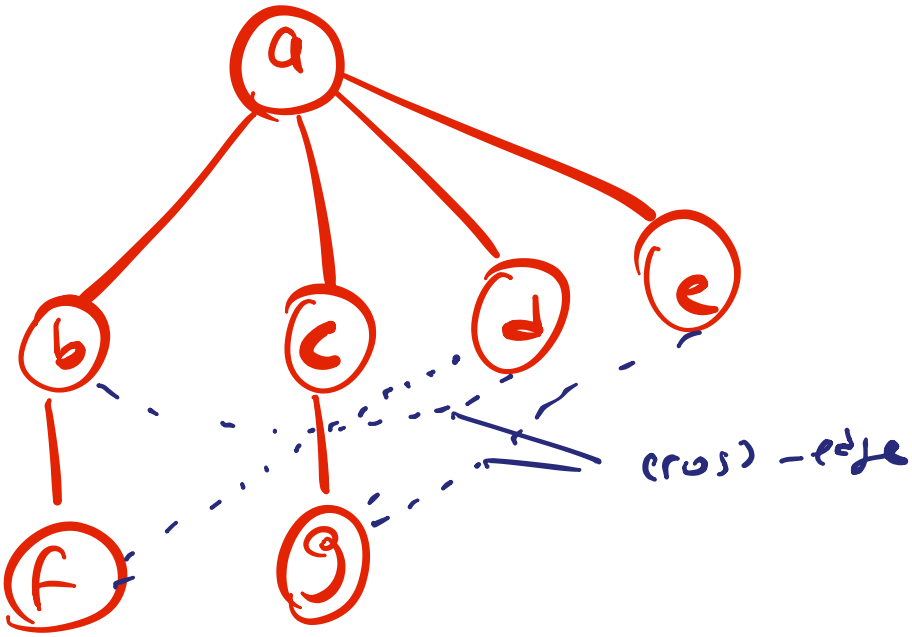
35

4. Traverse the graph of Problem 1 by breadth-first search and construct the corresponding breadth-first search tree. Start the traversal at vertex *a* and resolve ties by the vertex alphabetical order.



BFS - Queue

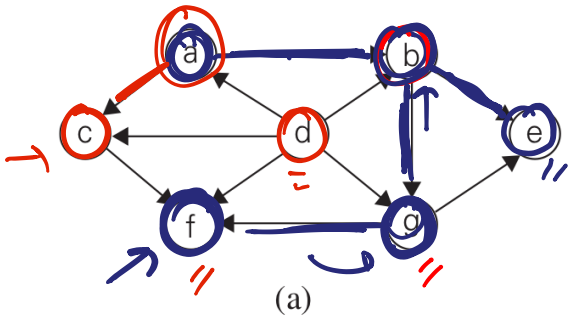
a b c d e f g



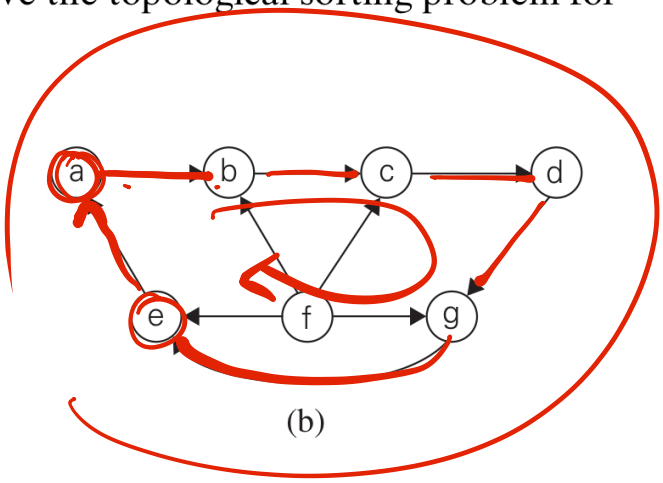
BFS

4.2

1. Apply the DFS-based algorithm to solve the topological sorting problem for the following digraphs:

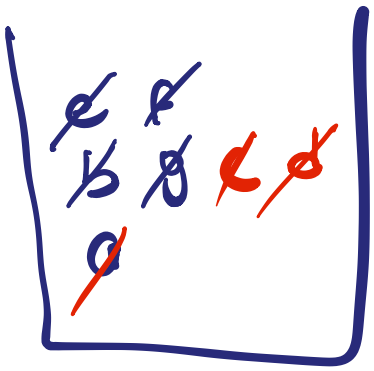


(a)



(b)

DFS : stack



Pop list

e f g b c a d

→ Reverse it to get topological order

d a c b g f e

b) Not DAG

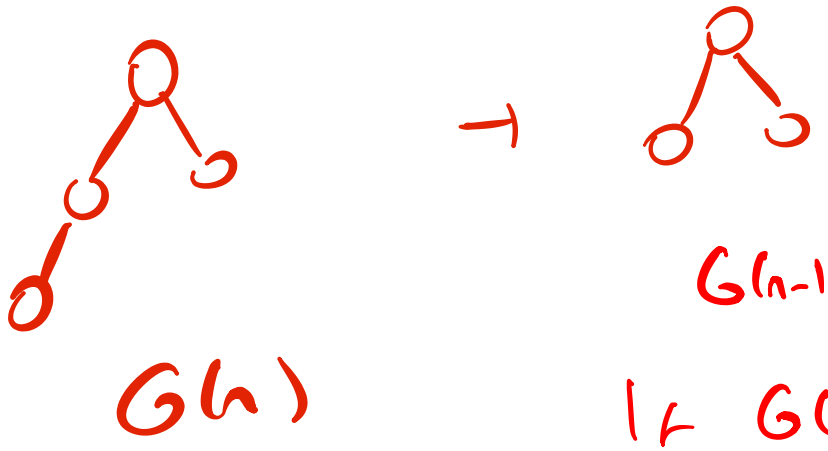
4.1

5. Consider the following algorithm to check connectivity of a graph defined by its adjacency matrix.

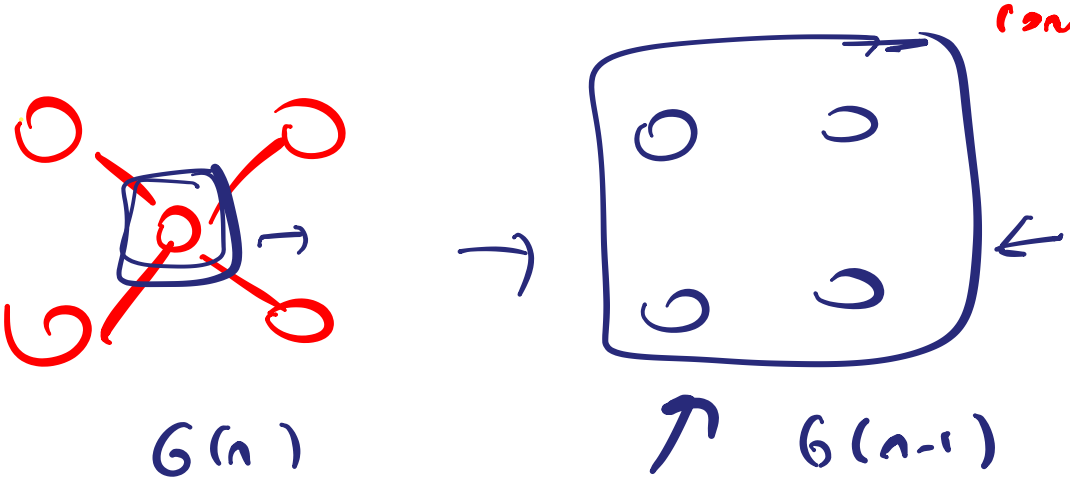
ALGORITHM *Connected*$(A[0..n-1, 0..n-1])$

```
//Input: Adjacency matrix A[0..n − 1, 0..n − 1]) of an undirected graph G
//Output: 1 (true) if G is connected and 0 (false) if it is not
if n = 1 return 1   //one-vertex graph is connected by definition
else
    if not Connected(A[0..n − 2, 0..n − 2]) return 0
    else for j ← 0 to n − 2 do
            if A[n − 1, j] return 1
        return 0
```

Does this algorithm work correctly for every undirected graph with $n > 0$ vertices? If you answer yes, indicate the algorithm's efficiency class in the worst case; if you answer no, explain why.



If G(n-1) is connected, G(n) is also connected



→ Not always true

4.3

7. Write pseudocode for a recursive algorithm for generating all 2^n bit strings of length n.

$n = 1$ $\{0, 1\}$

$n = 2$ $\{00, 01, 10, 11\}$

$n = 3$ $\{000, 001, 010, \ldots\}$

Algorithm BSR (n) {

// Input : positive integer

// output : Combination of bit strings

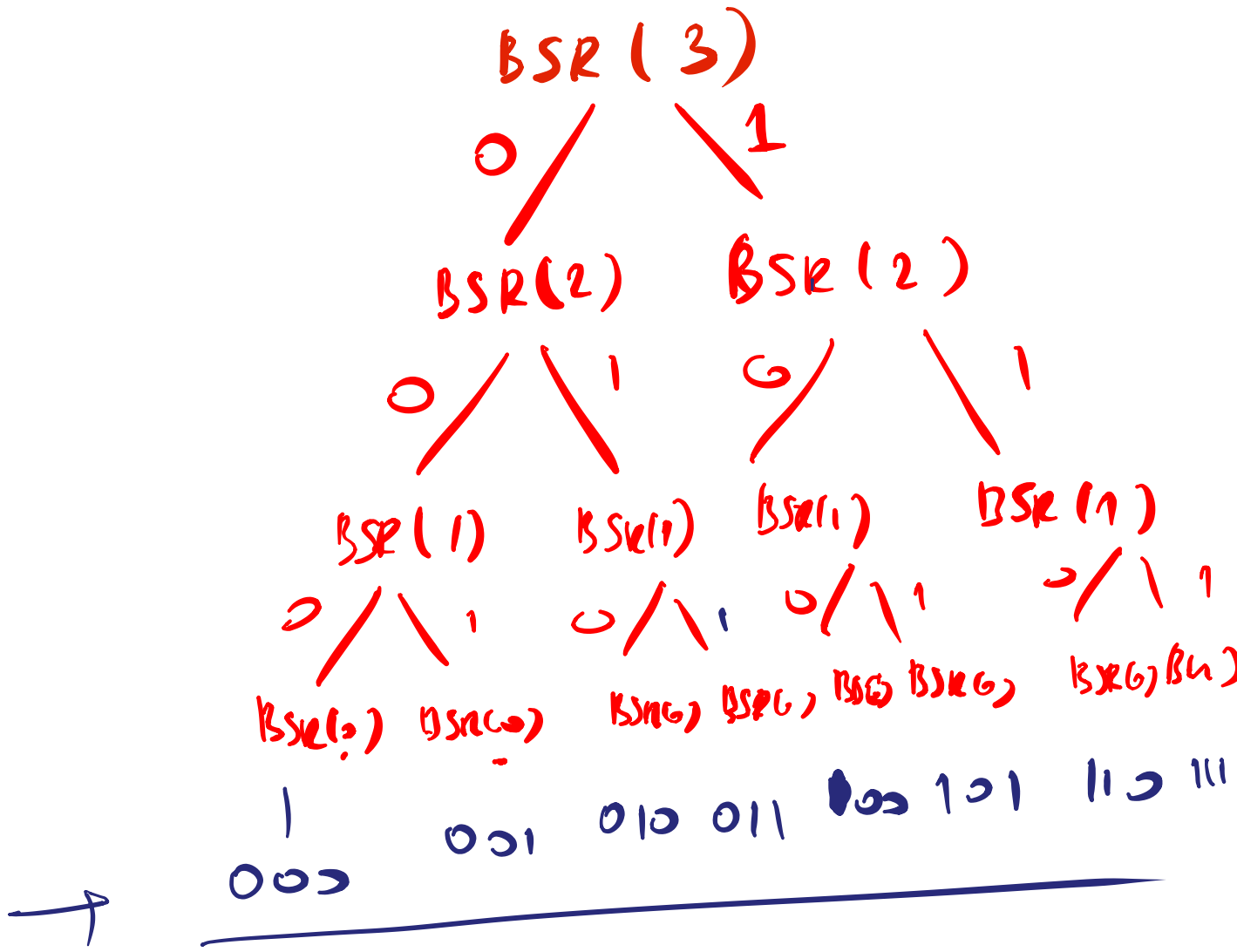
If n == 0

print (B)

else B[n-1] ← 0 ; BSR (n-1)

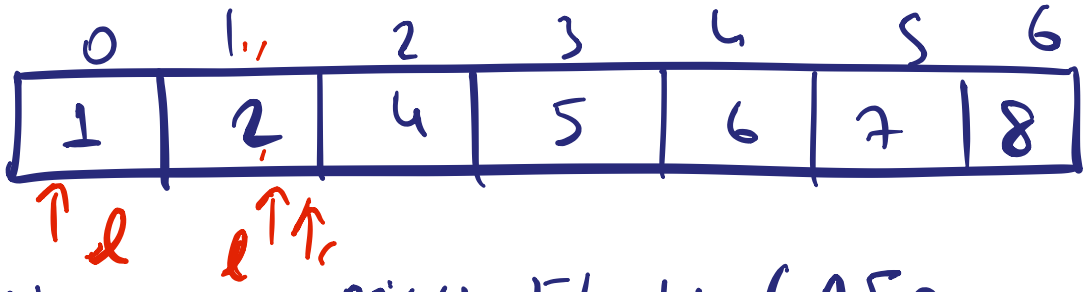
B[n-1] ← 1 ; BSR (n-1)

}


BSR(3)
0
1
BSR(2)
BSR(2)
0
1
0
1
BSR(1)
BSR(1)
BSR(1)
BSR(1)
000
001
010
011
101
111

4.4

9. An array $A[0..n-2]$ contains $n-1$ integers from 1 to n in increasing order. (Thus one integer in this range is missing.) Design the most efficient algorithm you can to find the missing integer and indicate its time efficiency.



Algorithm missing Elmts (A[0, ... n-2])

l ← 0

r ← n-2

while l < r do {

m = ⌊ (l+r)/2 ⌋ // mdl elm

If A[m] == m+1

l ← m+1 // miss elmt in the right part

else r ← m-1 // missing elmt in the left side

If A[l] == l+1

return l+2

else return l+1

4.4

2. Design a decrease-by-half algorithm for computing $\lfloor \log_2 n \rfloor$ and determine its time efficiency.

$\log_2 4 = 2$

$n = 4$

$4/2 = 2$ } 2 division

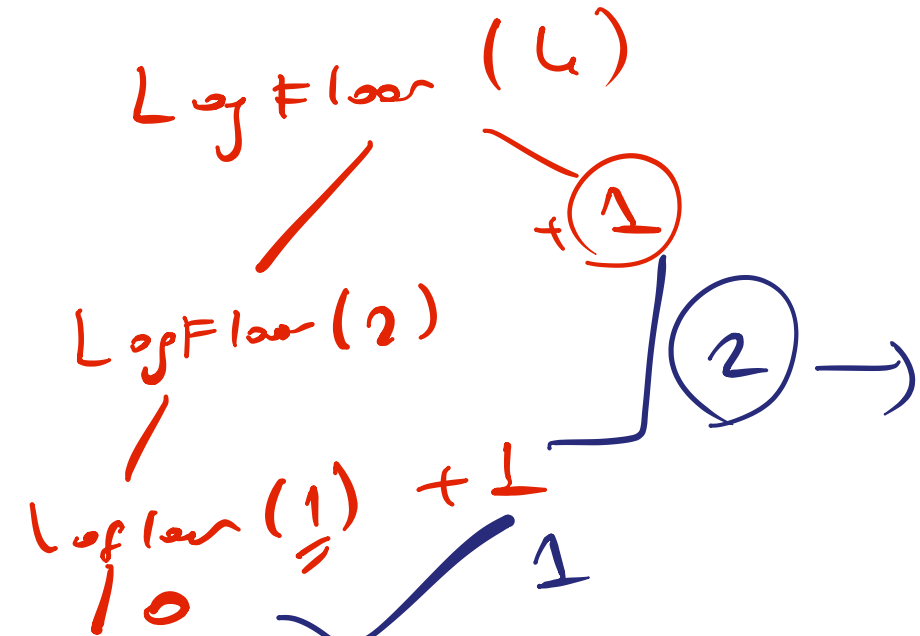
$2/2 = 1$

Algorithm LogFloor(n)

if n == 1 return 0

else

return LogFloor($\lfloor n/2 \rfloor$) + 1



of additions

$A(n) = A(\lfloor n/2 \rfloor) + 1$, for $n > 0$

$A(1) = 0$

$\Theta(\log n)$

4.4

3. a. What is the largest number of key comparisons made by binary search in searching for a key in the following array?

0	1	2	3	4	5	6	7	8	9	10	11	12
3	14	27	31	39	42	55	70	74	81	85	93	98

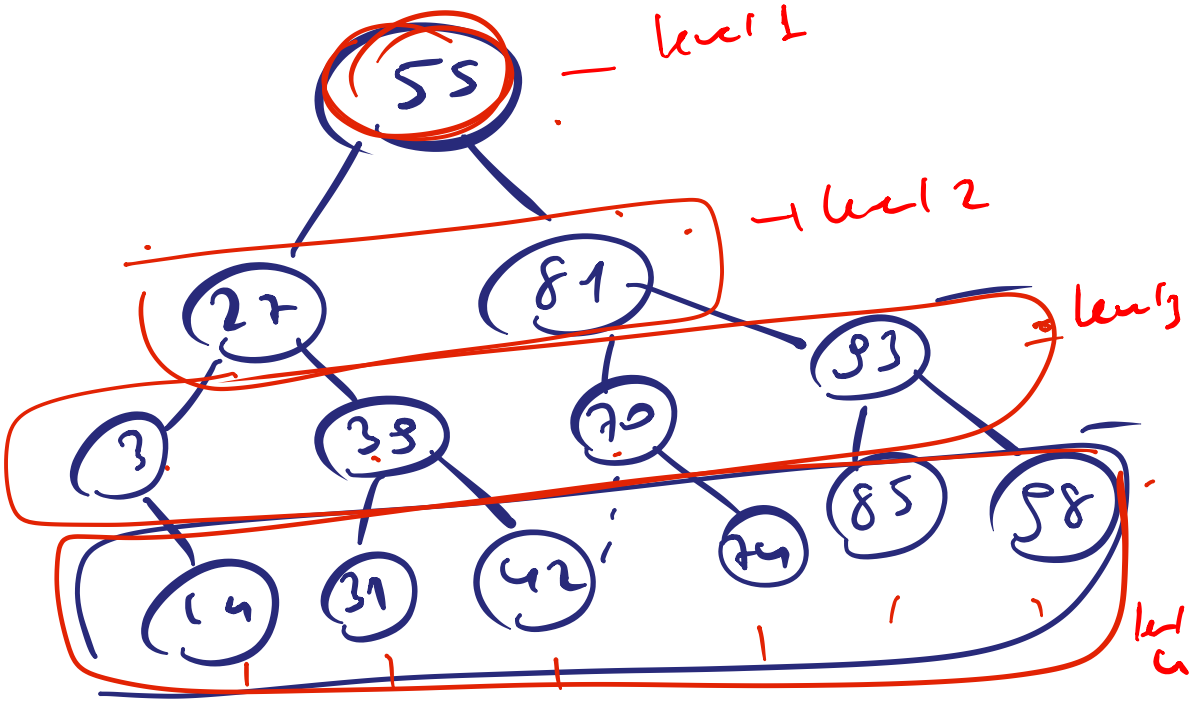
b. List all the keys of this array that will require the largest number of key comparisons when searched for by binary search.

c. Find the average number of key comparisons made by binary search in a successful search in this array. Assume that each key is searched for with the same probability.

a) $n = 13$

$\lfloor \log_2 13 \rfloor + 1 = 4$

b)



$$C_{avg} = \frac{1.1 + 2.2 + 3 \times 4 + 4 \times 6}{13}$$

$$\frac{2 + 4 + 12 + 24}{13} \approx 3.2$$